Manual Técnico

ContactoExtraterrestr3D



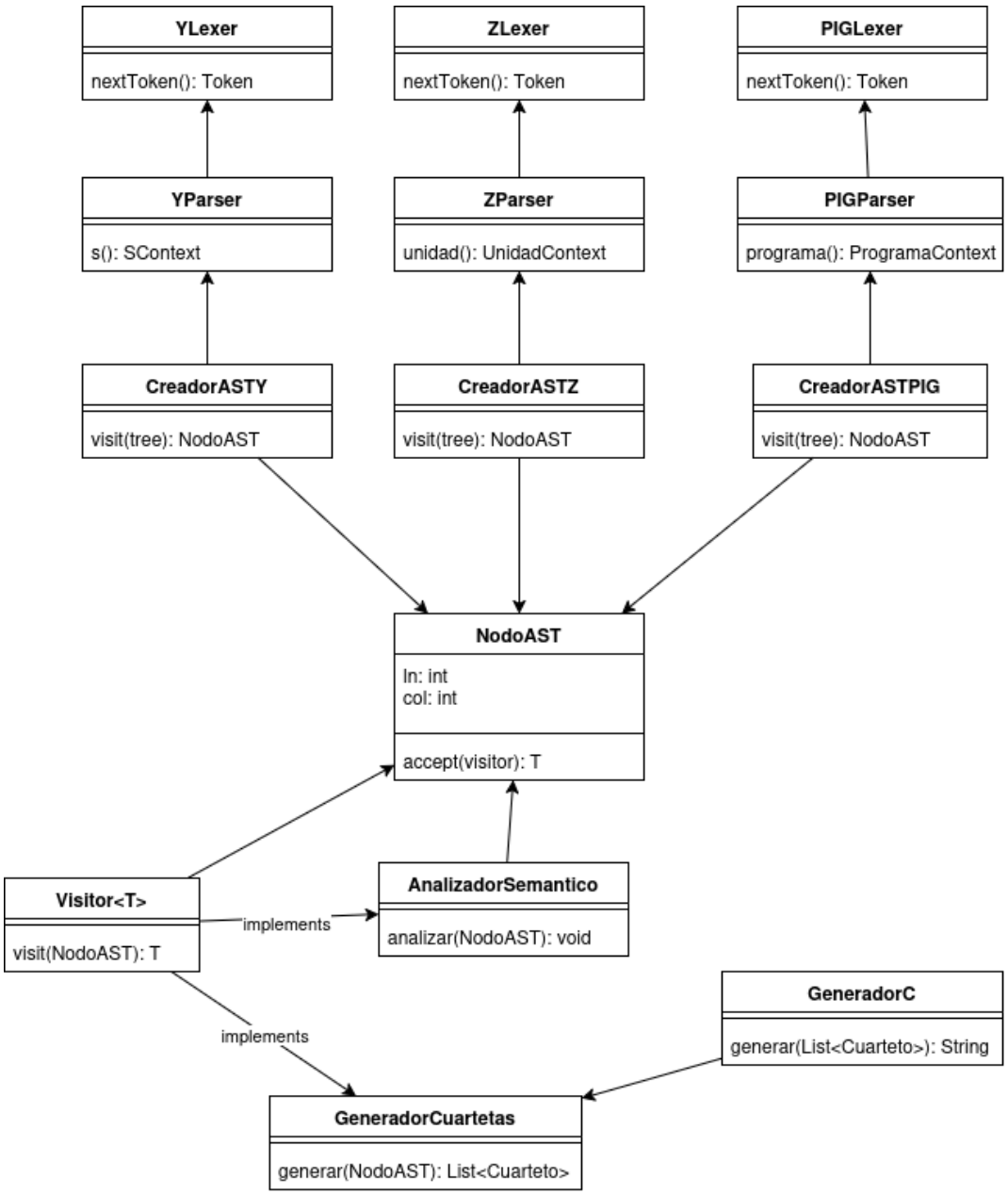
Introducción

Análisis de tres lenguajes de alto nivel para la construcción de código de tres direcciones y traducción a C.

Tecnologías utilizadas

Java 25
Antlr4 (4.13.2)
RSyntax 3.6.3

Diagrama de clases



Especificaciones de los lenguajes

Lenguaje Y

Palabras reservadas

Secciones de archivo:
    %estructuras %funciones

Definición de tipos / estructuras:
    estructura definir

Tipos primitivos:
    entero flotante caracter cadena bool

Literales booleanos:
    verdadero falso

Control de flujo:
    si entonces sino contrario
    elegir caso siempre romper
    para mientras hacer continuar

Funciones:
    retornar

Entrada/salida:
    imprimir leer

Símbolos y operadores

Aritméticos: + - * /
Relacionales: == != < >
Lógicos: && || !
Incremento/decremento: ++ --
Asignación: =
Delimitador de tipo array: [ ] (declaración: tipo nombre[N])
Acceso a miembro de estructura: .
Separador de parámetros: ,
Fin de definición de función/estructura/condición: :
Comentarios: // (línea) /* */ (bloque)
Bloques: definidos por INDENTACIÓN (tabulaciones), no por llaves.
Paso de parámetros compuestos (solo en firma de función):
    [] Tipo nombre -> arreglo pasado por referencia
    {} Tipo nombre -> estructura pasada por referencia

Precedencia de operadores (de mayor a menor):

1. ++ -- (postfijo/prefijo) 2. ! (negación unaria)
3. * / % 4. + -
5. < > == != 6. &&
7. || 8. ? :
9. = += -= *=

Lenguaje Z

Palabras reservadas

Estructura de clase:

public class

Tipos primitivos:

int double char boolean String
void (tipo de retorno vacío)

Literales booleanos:

true false
null (referencia nula de objeto)

Creación de objetos:

new

Control de flujo:

if else switch case default break
for while do continue
return

Entrada/salida:

println print readln

Símbolos y operadores

Aritméticos: + - * / %
Relacionales: == != < >
Lógicos: && || !
Incremento/decremento: ++ --
Asignación simple y compuesta: = += -= *=
Ternario: ? :
Acceso a miembro: .
Arreglos: [ ] multidimensional: [ ][ ]
Bloques: { }
Fin de sentencia: ;
Comentarios: // (línea) /* */ (bloque)
Comparación de objetos: ==

Precedencia de operadores (de mayor a menor):

1. ++ -- (postfijo/prefijo) 2. ! (negación unaria)
3. * / % 4. + -
5. < > == != 6. &&
7. || 8. ? :
9. = += -= *=

Lenguaje PIG

Palabras reservadas

Importación:
import

Marcadores de sección:
VARIABILES> MAIOR> FINIS;

Declaración:
esto series novus

Tipos primitivos:
numerus textum decimalis littera bool

Literales booleanos:
verum falsus

Control de flujo:
si aliter finis
dum facere per
perge interrumpe

Símbolos y operadores

Impresión (uno o encadenado): >>
Lectura (con o sin almacenar): <<
variable << -> lee y guarda en variable
<< -> solo lee, no almacena
Aritméticos: + - * /
Relacionales: >=
Lógicos: && ||
Asignación: =
Declaración de tipo: : (esto nombre : tipo valor;)
Acceso a miembro: .
Arreglos: [ ]
Bloques: { }

Fin de sentencia: ;
Comentarios: ## ... ##
Instanciación de objeto: Tipo novus Tipo(args)

Precedencia de operadores (de mayor a menor):

1. ++ --
2. * /
3. + -
4. < > >= <= ==
5. &&
6. ||
7. =

Tabla de compatibilidad de tipos

Y

Operación	Resultado
entero + entero	entero
entero + flotante	flotante
flotante + flotante	flotante
entero / entero	entero
entero / flotante	flotante
caracter + entero	entero
cadena + cadena	cadena
cadena + entero / flotante	cadena
bool && bool / bool || bool	bool
entero|flotante ==,!=,<,> mismo tipo	bool
entero ==,!=,<,> flotante	bool
estructura = estructura (mismo tipo)	estructura

Z

Operación	Resultado
int + int	int
int / int	int

int % int	int
double + int / int + double	double
double + double	double
String + int	String
String + double	String
String + String	String
String + boolean	String
char + int	int
boolean && boolean / ||	boolean
objeto == null	boolean
objeto == objeto (misma clase)	boolean
primitivo == primitivo	boolean

PIG

Operación	Resultado
numerus + numerus	numerus
numerus * numerus	numerus
numerus / numerus	numerus
decimalis + numerus	decimalis
decimalis + decimalis	decimalis
textum + numerus	textum
textum + decimalis	textum
textum + textum	textum
textum + bool	textum
bool && bool / ||	bool
littera + numerus	numerus